

KW007 SDK v1.3

- [Keywave KW007 — 5.8 GHz Doppler Radar Motion Sensor](#)
 - [Overview](#)
 - [Hardware Setup \(Critical\)](#)
 - [Quick Start](#)
 - [I²C Address](#)
 - [Supported Platforms](#)
 - [Detection Range Presets](#)
 - [Sensitivity](#)
 - [Initialization Sequence](#)
 - [Verifying a Write](#)
 - [Error Codes](#)
 - [Version](#)
 - [Module Reference](#)
- [API Reference](#)
 - [Application API — `kw007\_api.h`](#)
 - [Register-level interface — `kw007\_reg.h`](#)
 - [HAL interface — `kw007\_hal.h`](#)

Single-file reference. Everything is inline, so this opens from an email attachment, a share or a USB stick with nothing to unblock. The browsable version with a tree view and search is in `Docs/html/`.

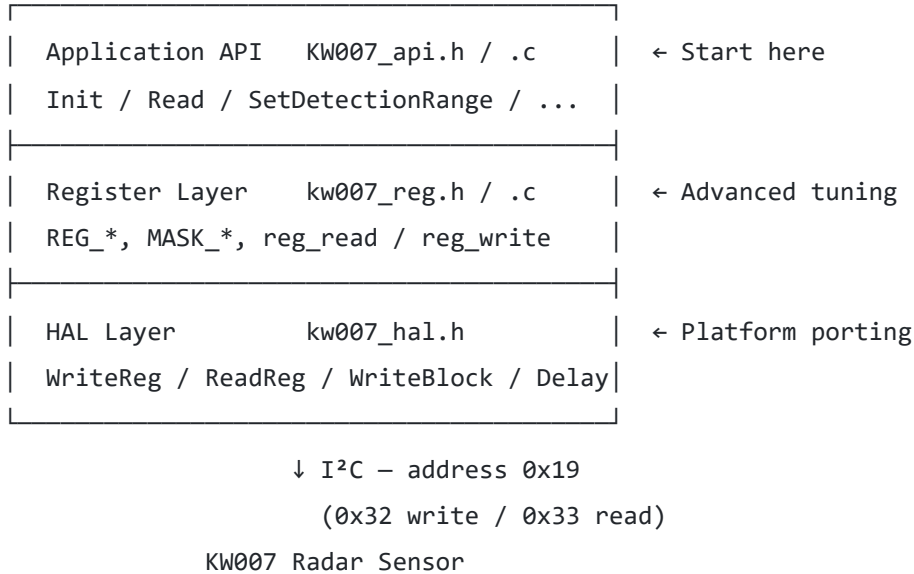
Keywave KW007 — 5.8 GHz Doppler Radar Motion Sensor

Release Date: 2026-09-07

High-level C SDK for application developers. No register knowledge required.

Overview

The KW007 SDK provides a clean, portable three-layer architecture:



No dynamic memory, no floating point, no recursion and no file-scope state. C99, depending only on `stdint.h`, `stdbool.h` and `string.h`.

Hardware Setup (Critical)

⚠ Before applying power, the **Range1** pin MUST be connected directly to GND, with no series resistor. A resistor on Range1 is a range setting; 0 Ω is the combination that selects I²C control.

⚠ Use either the 3.3 V or the 5 V input, never both, and never apply 5 V to the 3.3 V input — doing so will damage the module.

This switches the KW007 into I²C / software control mode. If Range1 is left floating, or tied through a resistor, the sensor runs in hardware (pin-select) mode and silently ignores all register writes — `kw007_Init()` will appear to succeed but no API call will have any effect.

Minimum wiring:

KW007 Pin	Connect to	Notes
VDD	3.3 V or 5 V	3.3 V recommended; see the warning above
GND	GND	
SDA	MCU SDA	Pull-up required
SCL	MCU SCL	Pull-up required
Range1	GND	Direct, no series resistor; mandatory for software control
GPO	MCU GPIO	Optional — hardware motion flag output

Quick Start

```
#include "KW007_api.h"

void setup(void) {
    KW007_Init();                // apply the start-up configuration
    KW007_SetDetectionRange(4);  // ~3 m detection
    KW007_SetSensitivity(2);     // lower = more sensitive
}

void loop(void) {
    KW007_Data d = KW007_Read();
    if (!d.ok) return;           // I2C failure
    if (d.approaching) { /* ... */ }
    if (d.leaving) { /* ... */ }
}
```

A complete, compilable example is in `Examples/kw007_example_led.c`.

I²C Address

Per the I²C-bus specification a device address is 7 bits wide. The KW007 address is factory-fixed, so a bus carries at most one KW007.

	Value
7-bit target address	0x19
First byte on the bus, write	0x32
First byte on the bus, read	0x33

Pass whichever form your I²C API expects: `KW007_I2C_ADDRESS_7BIT` (`0x19`) for Arduino Wire, Linux i2c-dev, smbus, ESP-IDF and Zephyr; `KW007_I2C_ADDRESS` (`0x32`) for STM32 HAL, bare-metal drivers and USB-I²C bridges.

Supported Platforms

Platform	HAL File
ESP32 / ESP32-S3 / ESP32-C3	User_platform/kw007_hal_arduino.cpp
Arduino AVR / SAMD / nRF52	User_platform/kw007_hal_arduino.cpp
STM32 (HAL / LL)	User_platform/kw007_hal_stm32.c
Windows + FTDI	User_platform/kw007_hal_ftdi.cpp
Any MCU with I ² C	User_platform/kw007_hal_user.c (template)

See `kw007_hal.h` for the four functions you need to implement.

Detection Range Presets

Call `KW007_SetDetectionRange(1..8)` to set gain, threshold, sensitivity and enhanced mode in one call.

Register mapping:

Field	Register	Bits	Constant
amp1	R19	[7:5]	REG_AMP1_GAIN
amp2	R20	[2:0]	REG_AMP2_AMP3_GAIN
amp3	R20	[4:3]	REG_AMP2_AMP3_GAIN
Threshold	R27	[7:0]	REG_THRESHOLD
Sensitivity	R10	[3:0]	REG_SENSITIVITY
Enhanced	R24	[6]	REG_ENHANCED_MODE

Preset table:

Range	amp1	amp2	amp3	Threshold	Sensitivity	Enhanced	~Distance
1	4	0	0	80	2	off	~20 cm
2	6	0	0	80	2	off	~50 cm
3	4	5	0	80	2	off	~1 m
4	7	4	0	80	4	off	~3 m
5	7	7	0	90	8	off	~5 m
6	7	7	1	64	8	off	~7 m
7	7	7	3	32	8	off	~10 m
8	7	7	3	32	8	on	~12 m

Ranges 1–3 reproduce what the sensor does in hardware (pin-select) mode, so a design that moves from the range pins to I²C keeps the same behaviour. Ranges 4–8 go beyond anything the pins can select.

The sensitivity is part of the preset, so call `KW007_SetSensitivity()` *after* this function to fine-tune it. Calling it first has no effect — the preset overwrites it.

The settings are written one after another rather than as a single transaction. A failure part way through leaves the earlier writes in place and returns the HAL's status.

Test conditions: distances measured with a stable power supply, no physical obstruction, and a human target. Unstable VDD or RF interference can significantly reduce the effective range.

Sensitivity

`KW007_SetSensitivity(0..15)` sets how much movement is needed before the sensor reports motion.

Field	Register	Bits	Constant
Sensitivity	R10	[3:0]	REG_SENSITIVITY , MASK_SENSITIVITY
Held at 0	R10	[5:4]	MASK_SENSITIVITY_RSVD

Higher value = less sensitive. Level 0 responds to the smallest movement and level 15 needs the largest; the sixteen steps are evenly spaced.

This is a separate control from `KW007_SetThreshold()`. The threshold works on how strong the reflection is; sensitivity works on how much the target moved. A target that is close but barely moving is rejected by the sensitivity setting, not by the threshold.

`KW007_Init()` leaves the sensitivity at 8, and `KW007_SetDetectionRange()` sets it as part of its preset. To match what the sensor produces in hardware (pin-select) mode:

```
KW007_Init();
KW007_SetSensitivity(2);
```

Two constraints come from the hardware:

- The setting only takes effect after `KW007_Init()`. R10 is one of the registers the chip can source either from the factory fuse or from I²C, and `R09[5]` picks which — for R10 – R13 as a group. `KW007_Init()` sets that bit; before it does, writes to R10 are ignored.
- R10[5:4] is fixed in hardware, and connecting Range1 directly to GND — already mandatory for software control — fixes it at 0. `KW007_SetSensitivity()` therefore drives those bits to 0 on every call, so the register cannot disagree with the pin.

Initialization Sequence

`KW007_Init()` runs the official start-up sequence:

1. Write R09 with bit[5] cleared, so R01 returns the factory VCO bank
2. Read R01[5:0] — the factory-burned VCO bank, different on every part
3. Patch it into R12[5:0] and burst-write R00 – R30, with R09 bit[5] set to restore I²C control

Nothing else will work until this succeeds.

Verifying a Write

The configuration functions do not verify their own writes — a write the bus acknowledges is reported as success. Where a design needs to be sure a setting took effect, call `KW007_VerifyReg()` afterwards with the same mask the setting uses:

```
KW007_SetThreshold(90);
if (KW007_VerifyReg(REG_THRESHOLD, 0xFF, 90) != KW007_OK) { /* ... */ }
```

Error Codes

Defined in `kw007_hal.h`:

Code	Value	Meaning
KW007_OK	0x00	Success
KW007_ERR_I2C	0x10	I ² C bus failure
KW007_ERR_NO_DEVICE	0x20	Sensor did not ACK
KW007_ERR_PARAM	0x30	Invalid argument
KW007_ERR_VERIFY	0x40	Read-back did not match

Version

```
#if KW007_SDK_VERSION >= KW007_SDK_VERSION_AT(1, 3)
    KW007_SetSensitivity(2);
#endif
```

Module Reference

- `KW007_api.h` — Application API (recommended entry point)
- `kw007_reg.h` — Register-level interface for advanced tuning
- `kw007_hal.h` — HAL interface to implement per platform

API Reference

Taken from the headers, so it cannot drift from them. Comment bodies are shown as written: several carry tables and aligned columns that reflowing would destroy.

Application API — KW007_api.h

KW007_DelayDuration

```
typedef enum {  
    DELAY_SETTING1 = 0,    /* 00 - 16 s */  
    DELAY_SETTING2 = 1,    /* 01 - 32 s */  
    DELAY_SETTING3 = 2,    /* 10 - 1 s */  
    DELAY_SETTING4 = 3     /* 11 - 64 s */  
} KW007_DelayDuration;
```

GPO pin hold-on time after motion stops.

Encoded in R29 bits[2:1].

R29[2:1]	Hold time
00	16 s
01	32 s
10	1 s
11	64 s

KW007_Init

```
uint8_t KW007_Init(void);
```


Initialize sensor. Runs the official startup sequence.

@pre The Range1 pin MUST be connected directly to GND before power-up, with no series resistor, to enable I2C / software control mode. If Range1 is left floating, or tied through a resistor, the sensor runs in hardware (pin-select) mode and all register writes are silently ignored.

@return KW007_OK Success
@return KW007_ERR_I2C I2C bus failure
@return KW007_ERR_NO_DEVICE Sensor did not ACK (check wiring, Range1 pin, and power supply)

KW007_Read

```
KW007_Data KW007_Read(void);
```

Read sensor output. Sensor updates at ~8Hz; poll in main loop.
Returns data.ok = false on I2C failure.

KW007_SetDetectionRange

```
uint8_t KW007_SetDetectionRange(uint8_t range);
```

Set detection range 1 (close) ~ 8 (far).

Sets gain (amp1 / amp2 / amp3), threshold, sensitivity and enhanced mode.

Register mapping:

```

amp1      → R19[7:5]   (REG_AMP1_GAIN)
amp2      → R20[2:0]   (REG_AMP2_AMP3_GAIN)
amp3      → R20[4:3]   (REG_AMP2_AMP3_GAIN)
Threshold → R27        (REG_THRESHOLD)
Sensitivity → R10[3:0] (REG_SENSITIVITY)
Enhanced  → R24[6]     (REG_ENHANCED_MODE)

```

Ranges 1-3 reproduce what the sensor does in hardware (pin-select) mode, so switching a design from the range pins to I2C keeps the same behaviour. Ranges 4-8 go beyond anything the pins can select; their gains carry over unchanged from SDK v1.2, with only the threshold and sensitivity lowered.

Approximate detection distance (stable power supply, no obstruction, human target):

Range	amp1 R19[7:5]	amp2 R20[2:0]	amp3 R20[4:3]	Thresh R27	Sens R10	Enh R24	~Distance
1	4	0	0	80	2	off	~20 cm
2	6	0	0	80	2	off	~50 cm
3	4	5	0	80	2	off	~1 m
4	7	4	0	80	4	off	~3 m
5	7	7	0	90	8	off	~5 m
6	7	7	1	64	8	off	~7 m
7	7	7	3	32	8	off	~10 m
8	7	7	3	32	8	on	~12 m

The sensitivity is part of the preset, so call `KW007_SetSensitivity()` after this function to fine-tune it - calling it first has no effect, because the preset overwrites it.

The settings are written one after another rather than as a single transaction. A failure part way through leaves the earlier writes in place and returns the HAL's status.

Actual range varies with power-supply stability, physical obstruction, target size, and motion speed. Unstable VDD or RF interference can significantly reduce effective distance.

KW007_SetGain

```
uint8_t KW007_SetGain(uint8_t amp1, uint8_t amp2, uint8_t amp3);
```

Set amplifier gains directly.

amp1 : 0-7 → R19[7:5] (REG_AMP1_GAIN)

amp2 : 0-7 → R20[2:0] (REG_AMP2_AMP3_GAIN)

amp3 : 0-3 → R20[4:3] (REG_AMP2_AMP3_GAIN)

@return KW007_ERR_PARAM if any value is out of range.

KW007_SetThreshold

```
uint8_t KW007_SetThreshold(uint8_t threshold);
```

Set detection threshold. Higher value = less sensitive.

threshold : 0-255 → R27[7:0] (REG_THRESHOLD)

Choosing a value: the threshold is compared against the reflected signal strength, which the KW007 GUI plots on the same 0-255 scale. With the area empty the chart shows the noise floor; with a target moving at the maximum required distance it shows the level the threshold must stay under. Set it between the two. Changing the gain moves both readings, so re-check afterwards. The Programming Guide has the full procedure.

KW007_SetEnhancedMode

```
uint8_t KW007_SetEnhancedMode(bool enable);
```

Enable / disable enhanced range mode.

enable : true = ON, false = OFF → R24[6] (REG_ENHANCED_MODE)

KW007_SetGPO_DelayDuration

```
uint8_t KW007_SetGPO_DelayDuration(KW007_DelayDuration delay);
```

Set GPO output hold time after motion stops.

delay : KW007_DelayDuration → R29[2:1] (REG_GPO_DELAY, MASK_GPO_DELAY)

The four durations are listed with @ref KW007_DelayDuration.

KW007_SetSlowSpeedDetection

```
uint8_t KW007_SetSlowSpeedDetection(bool enable);
```

Enable / disable slow-speed motion detection.

enable : true = ON, false = OFF → R21[5] (REG_SLOW_SPEED)

@note Hardware logic is inverted: bit[5]=0 enables slow detect, bit[5]=1 disables it. This API hides the inversion.

KW007_SetFastSpeedDetection

```
uint8_t KW007_SetFastSpeedDetection(bool enable);
```

Enable / disable fast-speed motion detection.

enable : true = ON, false = OFF (default)
→ R29[0] (REG_GPO_DELAY, MASK_FAST_SPEED_DET)

KW007_SetSamplingRate

```
uint8_t KW007_SetSamplingRate(bool use500Hz);
```

Set system sampling rate.

use500Hz : false = 1 kHz (default), true = 500 Hz
→ R12[7] (REG_VCO_CTRL, MASK_SAMPLE_RATE)

KW007_SetSensitivity

```
uint8_t KW007_SetSensitivity(uint8_t level);
```

Set motion sensitivity.

level : 0 (most sensitive) - 15 (least)

→ R10[3:0] (REG_SENSITIVITY, MASK_SENSITIVITY)

Higher value = less sensitive. Level 0 responds to the smallest movement and level 15 needs the largest; the sixteen steps are evenly spaced.

This is a separate control from KW007_SetThreshold(). The threshold works on how strong the reflection is; this works on how far the target moved. A target that is close but barely moving is rejected here, not there.

KW007_Init() leaves the sensitivity at 8, and

KW007_SetDetectionRange() sets it as part of its preset - 2 for ranges 1-3, 4 for range 4, 8 for ranges 5-8. Call this function after the range, not before, or the preset overwrites it.

@pre KW007_Init() must have run. It sets R09[5], which is what routes R10 to the I2C register instead of the factory fuse; before that write this setting has no effect.

@note The call also holds R10[5:4] at 0. That field is fixed in hardware, and tying Range1 to GND - already mandatory for software control - fixes it at 0. A register value that disagreed with the pin would not be honoured.

@return KW007_ERR_PARAM if level is greater than 15.

Register-level interface — kw007_reg.h

kw007_reg_write

```
uint8_t kw007_reg_write(uint8_t reg, uint8_t value);
```

Write single register via I2C

kw007_reg_read

```
uint8_t kw007_reg_read(uint8_t reg, uint8_t *value);
```

Read single register via I2C

kw007_reg_write_block

```
uint8_t kw007_reg_write_block(uint8_t startReg, uint8_t *data, uint8_t len);
```

Write consecutive registers (burst)

kw007_reg_rmw

```
uint8_t kw007_reg_rmw(uint8_t reg, uint8_t clearMask, uint8_t setBits);
```

Read-Modify-Write: clear clearMask bits, then set setBits

KW007_WriteReg

```
uint8_t KW007_WriteReg(uint8_t reg, uint8_t value);
```

Write register directly.

Valid range is R00-R30. Do not write R31 - see KW007_REG_COUNT. This function does not check, because an engineer API that silently refuses a write is worse than one that does as it is told; the caller is responsible for staying inside the range.

KW007_ReadReg

```
uint8_t KW007_ReadReg(uint8_t reg, uint8_t *value);
```

Read register directly. Result stored in *value.

KW007_VerifyReg

```
uint8_t KW007_VerifyReg(uint8_t reg, uint8_t mask, uint8_t expected);
```

Read a register back and compare the bits selected by mask.

The configuration functions do not verify their own writes - a write that the bus acknowledges is reported as success. Where a design needs to be sure a setting took effect, call this afterwards with the same mask the setting uses, for example:

```
KW007_SetThreshold(90);  
if (KW007_VerifyReg(REG_THRESHOLD, 0xFF, 90) != KW007_OK) { ... }
```

```
@return KW007_OK           the selected bits match  
@return KW007_ERR_VERIFY   they do not  
@return other              the error reported by the HAL
```

HAL interface — kw007_hal.h

KW007_HAL_WriteReg

```
uint8_t KW007_HAL_WriteReg(uint8_t devAddr, uint8_t reg, uint8_t data);
```

Write a single register.

devAddr : first byte on bus, 0x32 (7-bit address 0x19)

reg : register address

data : value to write

returns : 0 = success, non-zero = error

KW007_HAL_ReadReg

```
uint8_t KW007_HAL_ReadReg(uint8_t devAddr, uint8_t reg, uint8_t *data);
```

Read a single register.

devAddr : first byte on bus, 0x32; the read form 0x33 is derived here

reg : register address

data : pointer to store result

returns : 0 = success, non-zero = error

KW007_HAL_WriteBlock

```
uint8_t KW007_HAL_WriteBlock(uint8_t devAddr, uint8_t startReg, uint8_t *data, uint8_t len);
```

Write consecutive registers (burst write).

devAddr : first byte on bus, 0x32 (7-bit address 0x19)

startReg : first register address

data : data array

len : number of bytes to write

returns : 0 = success, non-zero = error

KW007_HAL_DelayMs

```
void KW007_HAL_DelayMs(uint32_t ms);
```

Delay for the specified number of milliseconds.